Homework 9 - Discussion

For this Ben Olschar, 108021211678 and Frederik Hüttemann, 108021215247 cooperated with eachother. We did implement the code on our own, but agreed to using one version. The discussion and results are made in cooperation.

Task 1: CPU Scan

We have seen, that the CPU Scan algorithm does have a time complexity of O(n). This means, with increasing number of elements, our CPU-Avg-time should grow linearly. This can be seen in the plot as well as in the table. Therefore, the results are expected. Note, that we changed the algorithm from being inclusive to exclusive to match the GPU implementation algorithm.

Task 2: GPU efficient scanning algorithm

The goal of the GPU efficient scanning algoritm was to implement a kernel, which has the same time complexity as the CPU implementation. As the CPU runs in O(n), the GPU should be able to do the same. The *Bernt-Kung algorithm* tries to implement this behavior.
When looking at the results, we can observe that the GPU is significantly faster than the CPU overall. However, there is a noticeable "knick" (jump) in the GPU execution times between 500,000 and 600,000 elements: the time jumps from ~0.18ms to ~0.62ms. This step change occurs because the recursive scan algorithm requires multiple kernel launches with `cudaDeviceSynchronize()` calls between them. As the input size grows, the number of blocks increases (e.g., 1,954 blocks for 500k vs. 2,344 blocks for 600k elements), and at some threshold the GPU can no longer efficiently schedule all blocks concurrently or memory access patterns become less cache-friendly. After this transition, the execution time scales linearly again within each regime. Note that the GPU and CPU results use different time scales for visualization purposes.
What can also be observed is that the CPU actually has linear time complexity. The GPU implementation however doesn't have linear complexity. For N <= 500.000, the execution time stays at ~0.2 ms, for N >= 600.000, we have again some kind of constant speed of ~0.6 ms. We would expect, that this pattern repeats for larger N (N >1.000.000), so we again get a jump. Also possible could be, that for N >= 600.000 a linear time complexity could be observed. For this, additional tests could be run, to proof or disproof this behavior.